

EXPERIMENT-14

1. Algorithm:

Algorithm

1. Start.
2. Declare variables "num", "temp", "choice", "digit", "i", an array "freq[10]", and a variable "unique".
3. Initialize all elements of "freq[10]" to "0" and "unique = 1".
4. Input a positive integer "num".
5. If "num <= 0", display ""Please enter a positive integer"" and stop.
6. Display the menu:
 - 1. Check whether all digits are unique.
 - 2. Find the frequency of each digit from 0 to 9.
7. Input the user's choice.
8. Copy "num" into "temp".
9. Use "switch(choice)" to perform the selected operation.

Case 1: Check whether all digits are unique

10. Repeat while "temp > 0":
 - Find the last digit using "digit = temp % 10".
 - Increase "freq[digit]" by 1.
 - If "freq[digit] > 1", set "unique = 0" and stop the loop.
 - Remove the last digit using "temp = temp / 10".
11. If "unique == 1", display ""Unique"".
12. Otherwise, display ""Not Unique"".

Case 2: Find frequency of each digit

13. Repeat while "temp > 0":
 - Find the last digit using "digit = temp % 10".
 - Increase "freq[digit]" by 1.
 - Remove the last digit using "temp = temp / 10".

14. Use a "for" loop from "i = 0" to "9".
15. Display the frequency of every digit as "freq_i".

Default Case

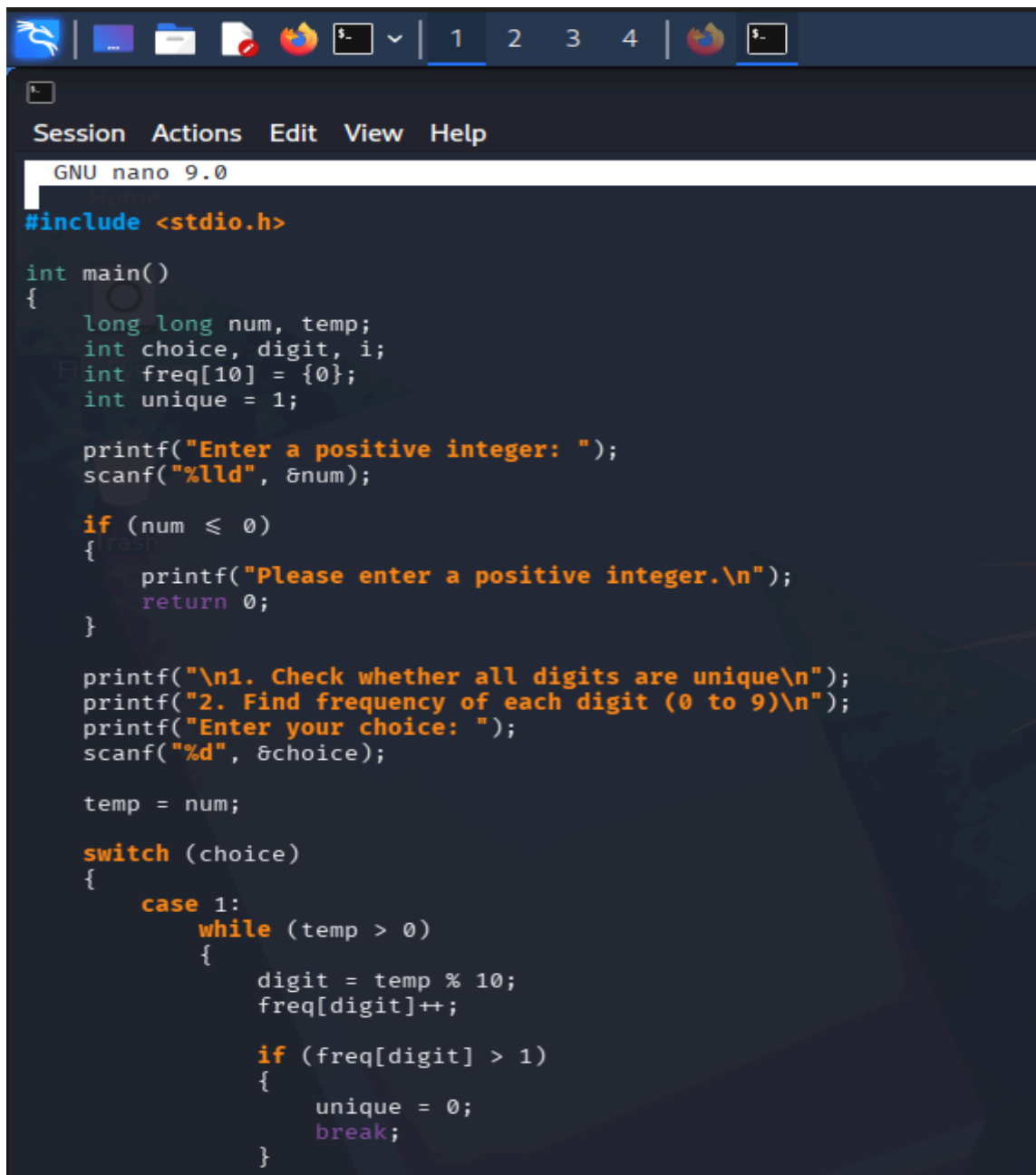
16. Display ""Invalid choice"" if the user enters a choice other than 1 or 2

17. Stop

2. Test Case Table

Test Case	Choice	Input Number	Expected Output	Actual Output	Result
1	1	12345	Unique	Unique	Pass
2	1	12234	Not Unique	Not Unique	Pass
3	1	9876543210	Unique	Unique	Pass
4	1	555	Not Unique	Not Unique	Pass
5	2	112333	freq_0=0, freq_1=2, freq_2=1, freq_3=3, others = 0	Same as expected	Pass
6	2	102030	freq_0=3, freq_1=1, freq_2=1, freq_3=1, others = 0	Same as expected	Pass

3. Program, Compile and Execution:



```
GNU nano 9.0
#include <stdio.h>

int main()
{
    long long num, temp;
    int choice, digit, i;
    int freq[10] = {0};
    int unique = 1;

    printf("Enter a positive integer: ");
    scanf("%lld", &num);

    if (num <= 0)
    {
        printf("Please enter a positive integer.\n");
        return 0;
    }

    printf("\n1. Check whether all digits are unique\n");
    printf("2. Find frequency of each digit (0 to 9)\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    temp = num;

    switch (choice)
    {
        case 1:
            while (temp > 0)
            {
                digit = temp % 10;
                freq[digit]++;

                if (freq[digit] > 1)
                {
                    unique = 0;
                    break;
                }
            }
        
```

```
        temp = temp / 10;
    }

    if (unique == 1)
        printf("Unique\n");
    else
        printf("Not Unique\n");

    break;

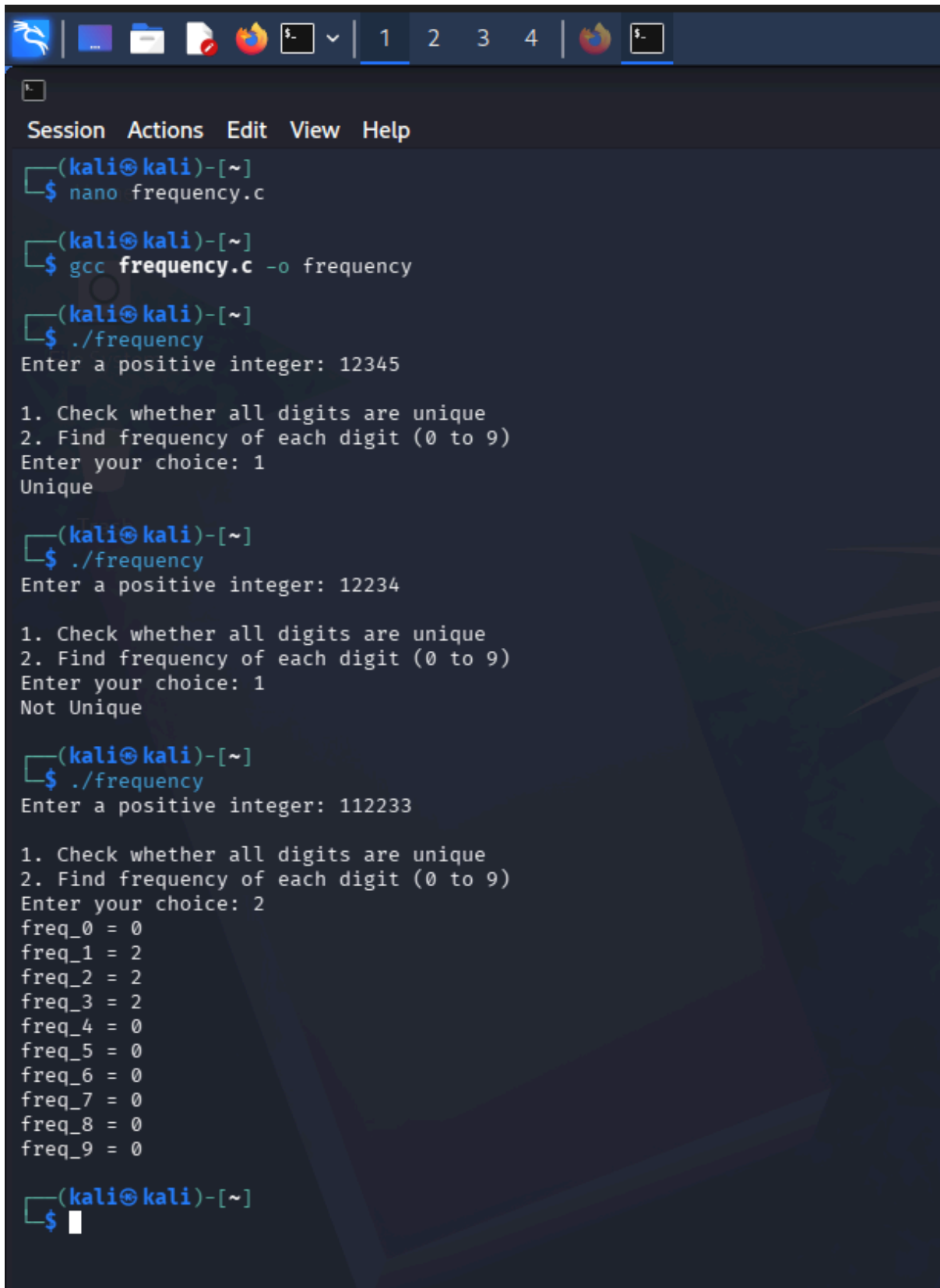
case 2:
    while (temp > 0)
    {
        digit = temp % 10;
        freq[digit]++;
        temp = temp / 10;
    }

    for (i = 0; i ≤ 9; i++)
    {
        printf("freq_%d = %d\n", i, freq[i]);
    }

    break;

default:
    printf("Invalid choice\n");
}

return 0;
}
```



```
(kali㉿kali)-[~]  
$ nano frequency.c  
  
(kali㉿kali)-[~]  
$ gcc frequency.c -o frequency  
  
(kali㉿kali)-[~]  
$ ./frequency  
Enter a positive integer: 12345  
  
1. Check whether all digits are unique  
2. Find frequency of each digit (0 to 9)  
Enter your choice: 1  
Unique  
  
(kali㉿kali)-[~]  
$ ./frequency  
Enter a positive integer: 12234  
  
1. Check whether all digits are unique  
2. Find frequency of each digit (0 to 9)  
Enter your choice: 1  
Not Unique  
  
(kali㉿kali)-[~]  
$ ./frequency  
Enter a positive integer: 112233  
  
1. Check whether all digits are unique  
2. Find frequency of each digit (0 to 9)  
Enter your choice: 2  
freq_0 = 0  
freq_1 = 2  
freq_2 = 2  
freq_3 = 2  
freq_4 = 0  
freq_5 = 0  
freq_6 = 0  
freq_7 = 0  
freq_8 = 0  
freq_9 = 0  
  
(kali㉿kali)-[~]  
$
```

Result: PASS

The program therefore works correctly for both menu operations and matches the examples given in the question.

NAME: TAANISH AGARWAL

BATCH: B-15

SAP ID: 590034662